

Security & Authentication (OWASP)

HttpOnly JWT cookies, bcrypt password hashing, CORS origin whitelisting, NoSQL injection sanitization, and multi-tier rate limiting — a defense-in-depth security model.

🔑 Auth: HttpOnly JWT Cookie 🔒 OWASP A01–A10 Compliance ⚙️ Rate: 3-Tier Limiting

1. Defense-in-Depth Security Architecture

🔒 OWASP-ALIGNED SECURITY LAYERS — HTTPONLY VS LOCALSTORAGE ATTACK SURFACE

❌ LocalStorage (Rejected)

XSS Attack Vector

Any injected `<script>` → `localStorage.getItem('token')`

- Token readable by any JS on the page
- 3rd party script injection = full account takeover
- CDN-served JS files are attack vector
- npm supply chain attacks steal tokens

OWASP A03: Injection Vulnerability

✅ HttpOnly Cookie (Our Choice)

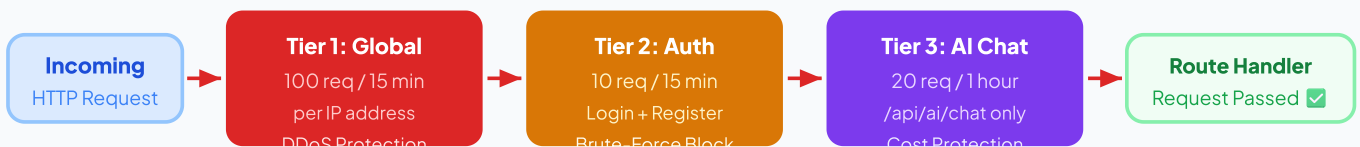
XSS-Immune by Design

HttpOnly flag → JavaScript cannot read the cookie

- Token invisible to all client-side JavaScript
- Browser auto-sends on every same-origin request
- SameSite=Strict blocks CSRF attacks
- Secure=true → HTTPS only transmission

OWASP A07: Auth Hardened ✅

🛡️ 3-TIER RATE LIMITING ARCHITECTURE — DDOS & BRUTE-FORCE DEFENSE



2. Technology Decision Framework

1. HttpOnly JWT Cookie vs LocalStorage

AUTH SECURITY

WHAT IS IT?

A browser HTTP-only cookie stores the JWT authentication token with flags that prevent JavaScript from accessing it.

HOW DOES IT WORK & PROBLEM SOLVED?

LocalStorage tokens are vulnerable to any XSS attack — an injected third-party script can call `localStorage.getItem('token')` and steal the session. HttpOnly cookies are inaccessible to JavaScript by browser design. Only the browser's network layer can read and send them.

WHAT ALTERNATIVES EXISTED?

LocalStorage: Simpler to implement, but OWASP A07 non-compliant. **SessionStorage:** Same XSS vulnerability as LocalStorage.

WHY DID WE CHOOSE THIS APPROACH?

2. bcrypt vs SHA-256 for Password Hashing

CREDENTIAL SAFETY

WHAT IS IT?

An adaptive hashing algorithm specifically designed for passwords that intentionally runs slowly and adds a cryptographic salt to prevent rainbow table and brute-force attacks.

HOW DOES IT WORK & PROBLEM SOLVED?

SHA-256 hashes a billion passwords per second on modern hardware. bcrypt is tunable via a "cost factor" (we use 12) that makes each hash take ~250ms. An attacker who steals our database would need 34 years to brute-force a 10-character password.

WHAT ALTERNATIVES EXISTED?

SHA-256: Extremely fast — useful for checksums, catastrophic for passwords. **MD5:** Cryptographically broken since 1996.

WHY DID WE CHOOSE THIS APPROACH?

bcrypt is the OWASP-recommended standard for password storage. Cost factor 12 provides a 250ms computation time — imperceptible to a user logging in, but 34 years for an attacker brute-forcing offline.

💡 Senior Interview Defense: "How do you secure user authentication?"

"UniCoach uses a defense-in-depth authentication model. JWTs are stored in HttpOnly cookies — never LocalStorage — making them immune to XSS token theft. Passwords are hashed with bcrypt at cost factor 12, giving 250ms compute time per attempt and making brute-force attacks computationally infeasible. A 3-tier rate limiter blocks credential stuffing at the auth endpoints and protects AI endpoints from cost exhaustion. All inbound data is sanitized against NoSQL injection via express-mongo-sanitize before touching any MongoDB query."